

Name: _____

Class: _____



Mr. Rawson • GCSE Computer Science

Year 10 2026

Binary Shifts

Work in pencil. Every bit moves the same number of columns, all together. The empty columns left behind are filled with 0. Anything pushed off either end is **lost**.

The two rules

Left shift — every bit moves *left*.

Each column left is worth *twice* as much, so a shift of n places **multiplies by 2^n** .

1 place $\times 2$ 2 places $\times 4$ 3 places $\times 8$

Right shift — every bit moves *right*.

Each column right is worth *half* as much, so a shift of n places **divides by 2^n** .

1 place $\div 2$ 2 places $\div 4$ 3 places $\div 8$

Worked example 1 22, shifted LEFT 1 place

Place value	128	64	32	16	8	4	2	1	Denary
Number	0	0	0	1	0	1	1	0	22
Answer	0	0	1	0	1	1	0	0	44
Check			32		8	4			

The **Check** row is the proof: the 1s now sit in the 32, 8 and 4 columns, and $32 + 8 + 4 = 44$. The original was 22, so the shift has multiplied by 2.

Worked example 2 20, shifted RIGHT 1 place

Place value	128	64	32	16	8	4	2	1	Denary
Number	0	0	0	1	0	1	0	0	20
Answer	0	0	0	0	1	0	1	0	10
Check					8		2		

$8 + 2 = 10$, and the original was 20 — divided by 2.

Why this matters

Multiplying by 2 is the easiest thing to show you, but it is not what shifting is mainly for. **Shifting is how a computer takes a number apart.** One colour pixel packs red, green and blue into a single number, and a shift is how you pull one of them out — as do memory addresses, network data, file formats, hashing and encryption.

In hardware a shift is almost free: nothing is calculated, the bits are simply wired one column across. It is the fastest thing a processor can do. And a shift of 4 places moves the number exactly one **hexadecimal** digit — the nibble trick and the binary shift turning out to be the same fact.

Examiners' Report

The commonest mark lost on shifts is describing the *direction* but not the *number of places* — “shift left” instead of “shift left by one place”. Always give both.

Left shifts

1. Shift **13 LEFT** by **1** place.

Place value	128	64	32	16	8	4	2	1	Denary
Number	0	0	0	0	1	1	0	1	13
Answer	0	0	0	1	1	0	1	0	26
Check				16	8		2		

2. Shift **37 LEFT** by **2** places.

Place value	128	64	32	16	8	4	2	1	Denary
Number	0	0	1	0	0	1	0	1	37
Answer	1	0	0	1	0	1	0	0	148
Check	128			16		4			

3. Shift **100 LEFT** by **2** places.

Place value	128	64	32	16	8	4	2	1	Denary
Number	0	1	1	0	0	1	0	0	100
Answer	1	0	0	1	0	0	0	0	144
Check	128			16					

Right shifts

4. Shift **44 RIGHT** by **1** place.

Place value	128	64	32	16	8	4	2	1	Denary
Number	0	0	1	0	1	1	0	0	44
Answer	0	0	0	1	0	1	1	0	22
Check				16		4	2		

5. Shift **200 RIGHT** by **2** places.

Place value	128	64	32	16	8	4	2	1	Denary
Number	1	1	0	0	1	0	0	0	200
Answer	0	0	1	1	0	0	1	0	50
Check			32	16			2		

6. Shift **25 RIGHT** by **1** place.

Place value	128	64	32	16	8	4	2	1	Denary
Number	0	0	0	1	1	0	0	1	25
Answer	0	0	0	0	1	1	0	0	12
Check					8	4			

What went wrong

Questions 3 and 6 did not give the answer the arithmetic says they should. Both lost a bit — but off *opposite ends* of the grid, and that is the whole difference.

7. Look back at **question 3**. $100 \times 4 = 400$, but the grid does not say 400. What has happened, and what is it called? **Overflow**. 400 needs nine bits. The 1 that was in the 64s column should have moved to a 256s column, and there is not one — it was pushed off the LEFT end and lost. The grid reads 144, which is $400 - 256$.

8. Look back at **question 6**. $25 \div 2 = 12.5$, but the grid does not say 12.5. What has happened here, and is it the same problem as question 3? **Loss of precision — and no, it is the opposite end**. The 1 in the 1s column was pushed off the RIGHT and lost. Binary integers cannot hold a half, so the fraction is thrown away and the answer is rounded down to 12. Question 3 lost a bit off the left, which made the number too big to store; this loses a bit off the right, which makes it not exact.

9. A programmer needs to multiply by 8 very quickly. Which shift should they use, and in which direction? **A left shift of 3 places, because $2^3 = 8$** .

What the computer actually does when it overflows

It does not stop, and it does not report an error. There are eight wires, so eight bits come out; the ninth has nowhere to go. It is put into a single **flag** inside the processor — a warning light, not a brake. The eight bits that remain *are* stored, and they are a perfectly valid 8-bit number. Just not the right one.

So $150 + 130$ is stored as **24**, and the program carries on using 24 as though it were correct. Unless it has been written to check that flag, nothing happens at all. **The silence is the dangerous part**. A clock does the same thing and nobody calls it broken: 11 o'clock plus 3 hours is 2 o'clock, not 14. It has run out of room and started again.

And shifting right too far? Every 1 falls off the end. Shift an 8-bit number right 8 places and the answer is **0**. Nothing breaks there either — the number is simply gone.